Fpt University

TRƯỜNG ĐẠI HỌC FPT

CAPSTONE PROJECT REPORT

Report 2 – Project Management Plan

– Hanoi, Sep 2026 –

Table of Contents

Change Log....3
I. Project Overview....4
1. Scope and Purpose....4
2. Assumptions & Constraints....4
3. Cost & Time Estimations....5
4. Capacity Planning....7
5. Project Objectives....7
6. Risks and Mitigations....7
II. Management Approach....8
1. Project Process/Methodology....8
2. Quality Management....9
2.1 Defect Prevention & Code Standards....9
2.2 Testing Levels & Validation....9
3. Project Training Plan....9
4. Responsibility Assignments....10
5. Project Communications....11
6. Configuration Management....11
6.1 Document Management....11
6.2 Source Code Management....11
6.3 Tools & Infrastructures....12

Change Log

Date	A* M, D	In charge	Change Description
30/9/2026	A	Nguyen Hoang Thang	Add I.Project Overview
30/9/2026	A	Nguyen Hoang Thang	Add II.Management Approach

*A - Added M - Modified D - Deleted

I. Project Overview

1. Scope and Purpose

The primary objective of the EcoLink project is to build an Intelligent Recyclable Waste Classification and Trade Matching System. The project addresses two core environmental challenges in Vietnam: a widespread lack of household waste-sorting knowledge and the absence of an efficient, economically viable channel connecting households with local scrap collectors and recycling facilities. By combining Computer Vision (AI) image recognition, localized sorting instructions, GPS-based location matching, and gamification incentives, EcoLink transforms routine waste disposal into an engaging, eco-friendly, and financially rewarding habit.

Architected as a responsive web application, EcoLink incorporates several key functional modules within its core scope. The system features AI-driven waste classification capable of identifying 8 to 12 common categories of recyclable waste using trained YOLOv8 or CNN models. This classification triggers a localized sorting rule engine that delivers dynamic, step-by-step cleaning and disposal instructions. Furthermore, households can list recyclable waste while discovering nearby verified recyclers through spatial querying using PostgreSQL/PostGIS. Trade coordination is managed end-to-end through a structured workflow—from initial request to acceptance, receipt confirmation, and completion or cancellation.

2. Assumptions & Constraints

EcoLink is an independent capstone project developed by group **SEP490-G141** during the Fall 2026 semester at FPT University. It does not inherit legacy codebases from prior university projects and operates as a standalone software platform.

No	Description	Note
Assumptions		
1	All 5 team members maintain dedicated availability (40 hours/week) throughout the 14-week project lifecycle.	Resources / Capacity
2	Lecturer / Supervisor (Mr. Ta Dinh Tien) reviews and provides feedback on milestone deliverables within 3–5 business days.	Review & Approval
3	Publicly available datasets (Kaggle Garbage 12-Class, TrashNet, TACO) supplemented by 40% self-collected real-world photos provide sufficient training quality for the AI model	Technical / AI Data
4	Open-source software components (.NET 10 SDK, Python FastAPI, PostgreSQL/PostGIS, Docker, MinIO, Redis) remain stable and royalty-free.	Technology Stack
Constraints		
1	The final software package, documentation, and oral defense must be completed strictly within the 14-week semester timeline (September 2026 – December 2026).	Schedule
2	Zero hardware/license budget; the system must be built and deployed using free-tier cloud infrastructure or local Docker containerization.	Budget

3	System core architecture must follow Clean Architecture (.NET) combined with a dedicated Python FastAPI AI Service.	Technical Architecture
4	The system must comply with standard web application security rules (role-based access control, hashed passwords, non-public storage for internal service transaction receipts).	Security

3. Cost & Time Estimations

#	Work Package	Effort (pds)	Due Date
1	**Stage 1: Project Initiating (Week 1)**	**20**	**13/09/2026**
1.1	Project Team Setup, Topic Selection & Supervisor Alignment	10	11/09/2026
1.2	Report 1: Project Introduction v1.0	10	13/09/2026
2	**Stage 2: Project Planning & Initial Requirements (Weeks 2–3)**	**40**	**27/09/2026**
2.1	AI Feasibility Survey & Medium Use Cases Definition (24 UCs)	10	15/09/2026
2.2	User & Recycler Requirements Survey (Google Forms & Field Work)	10	24/09/2026
2.3	Report 2: Project Management Plan v1.0	10	26/09/2026
2.4	Report 3: Software Requirement Specification (SRS v0.9)	10	27/09/2026
3	**Stage 3: Software Design & Proof of Concept (Weeks 4–5)**	**45**	**11/10/2026**
3.1	Vertical Prototype / AI Vision Service POC (YOLOv8 + FastAPI)	15	04/10/2026
3.2	Report 4: Software Design Specification (SDS v1.0 - Architecture & ERD)	15	08/10/2026
3.3	Report 5: Test Planning & Test Strategy	10	11/10/2026
3.4	Report 3: SRS v1.0 (Finalized Requirements for Iteration 1)	5	11/10/2026
4	**Stage 4: Implementation & Testing (Weeks 6–11)**	**140**	**22/11/2026**
4.1	*Iteration 1: Core AI Vision & Auth (Weeks 6–7)*	45	25/10/2026
4.1.1	Backend Base Architecture (.NET Clean Architecture) & User Auth	15	18/10/2026

4.1.2	AI Scan Service & Knowledge Base Integration (FE & BE)	20	25/10/2026
4.1.3	Unit Testing & Iteration 1 Software Package v1.0	10	25/10/2026
4.2	*Iteration 2: Rule Engine, GPS Recycler Discovery & Trade Requests (Weeks 8–9)*	45	08/11/2026
4.2.1	PostGIS Geo-Matching, Listing Management & Recycler Discovery	20	01/11/2026
4.2.2	Trade Coordination Workflow & Handover Receipts	15	08/11/2026
4.2.3	Integration Testing & Iteration 2 Software Package v2.0	10	08/11/2026
4.3	*Iteration 3: Gamification, Admin Portal, Staff Portal (Weeks 10–11)*	50	22/11/2026
4.3.1	Virtual Tree Gamification, Eco Facts & Leaderboards	15	15/11/2026
4.3.2	Admin Dashboard, AI Misclassification Feedback Loop & Dataset Curation	20	22/11/2026
4.3.3	System Integration Testing & Software Package v3.0	15	22/11/2026
5	**Stage 5: Verification & Validation (Weeks 12–13)**	**25**	**06/12/2026**
5.1	System Testing, Performance Testing & Bug Fixing	10	29/11/2026
5.2	Report 6: User Guides & Installation Manuals	10	03/12/2026
5.3	Final Software Package Freeze (Documents & Source Code Deployment)	5	06/12/2026
6	**Stage 6: Project Closing (Week 14)**	**10**	**13/12/2026**
6.1	Report 7: Final Project Report	5	10/12/2026
6.2	Slide Presentation & Final Capstone Defense Preparation	5	13/12/2026
TOTAL	**Entire Project Lifecycle (14 Weeks)**	**450 pds**	**13/12/2026**

4. Capacity Planning

Item	Value
Hours per person per week	56 h (8 h/day × 7 days)
Team total per week	280 h/week (5 people × 56 h/week)
Total duration	14 weeks
Gross capacity	3920 h (490 person-days)
Buffer (meetings, reviews, unplanned — 20%)	784 h (98 person-days)
Effective working capacity	3136 h (392 person-days)

5. Project Objectives

#	Metric	Unit	Planned	Actual	Notes / References
1	**Total Effort Usage**	Person-Day (pds)	≤ 280 pds	*TBD*	Measured against effective working capacity
2	**Requirement Completeness**	Percentage (%)	≥ 95%	*TBD*	Ratio of implemented use cases to approved SRS (Fail if < 75%)
3	**Schedule Timeliness**	Percentage (%)	100%	*TBD*	On-time deliverables / Total planned milestone deliverables
4	**AI Model Accuracy (Top-1)**	Percentage (%)	≥ 85%	*TBD*	Tested on 8–12 common recyclable waste categories
5	**AI Out-of-Distribution Rejection**	Confidence Threshold	< 60% Confidence	*TBD*	Prompts user verification for uncertain/non-waste objects
6	**Web Page Load Time**	Seconds (s)	≤ 2.0 s	*TBD*	Evaluated under up to 50 concurrent active users
7	**Review Defects (Severity 1 & 2)**	Number of defects	0 open	*TBD*	Critical defects found during supervisor reviews must be resolved
8	**System Test Defects Resolution**	Percentage (%)	100% High/Critical	*TBD*	All blocker defects must be fixed before final defense
1	**Total Effort Usage**	Person-Day (pds)	≤ 280 pds	*TBD*	Measured against effective working capacity

6. Risks and Mitigations

#	Risk	Prob	Impact	Mitigation

R01	**Low AI Accuracy for Ambiguous / Damaged Waste** *AI fails to correctly distinguish crumpled paper, dirty plastic, or complex items.*	Medium	High	**1.** Restrict initial model scope to 8–12 well-defined recyclable classes. **2.** Enforce a 60% confidence threshold to prompt user confirmation if uncertain. **3.** Supplement public datasets with 40% self-collected real-world local photos.
R02	**Network Latency / Service Outage during Live Demo** *Cloud AI API endpoint fails or lags during supervisor or board defense.*	Medium	High	**Convert trained model (TensorFlow.js / TFLite)** to run locally as an offline backup inside the client browser, ensuring the core recognition feature works without the internet.
R03	**Scope Creep & Late Requirement Changes** *Uncontrolled additions of complex features (e.g., online payments, native apps).*	Medium	High	**1.** Formally freeze SRS after Week 5 review. **2.** Explicitly list Exclusions (LI-01 to LI-10) in Report 1 & 3. **3.** Require impact analysis and supervisor approval for any Change Request.
R04	**PostGIS Spatial Matching Performance Bottleneck** *Slow response when searching for recyclers by GPS coordinates under heavy load.*	Low	Medium	**1.** Apply GiST spatial indexing on PostGIS location columns. **2.** Cache active recycler profiles and geographic coordinates in Redis In-Memory cache.
R05	**Insufficient Real-World User & Recycler Survey Data** *Difficulty reaching enough local scrap collectors (vựa ve chai) for survey data.*	Medium	Medium	**1.** Assign a dedicated 3-member sub-team (Luan, Duong, Hoang). **2.** Conduct direct field interviews at local scrap yards in Hanoi. **3.** Utilize online Google Forms targeting eco-conscious communities.

II. Management Approach

1. Project Process/Methodology

The EcoLink project applies a **Hybrid Software Development Methodology**, combining **Waterfall** for project initiation, requirement freezing, and design phases with **Agile/Scrum Iterative Development** for software implementation. This approach ensures structured compliance with academic milestones while maintaining flexibility during feature implementation.

Initiation & Requirement Planning Phase (Weeks 1–3):

- Establish core team roles, set up Git repositories, and align with Supervisor (Mr. Ta Dinh Tien).
- Conduct real-world user surveys, perform gap analysis against existing market solutions, and formalize the Software Requirement Specification (SRS v0.9/v1.0) and Project Management Plan (PMP).

Architecture & Detailed Design Phase (Weeks 4–5):

- Build a Vertical Prototype / Proof of Concept (POC) for the Python FastAPI YOLOv8 AI Vision Service.
- Construct System Design Specifications (SDS v1.0), ERD schemas, API inventories, and Test Plans.

Iterative Implementation Phase (Weeks 6–11):

- **Iteration 1 (Weeks 6–7):** Core Backend setup (.NET Clean Architecture), JWT Authentication, AI Image Classification API, and Rule Engine mapping.
- **Iteration 2 (Weeks 8–9):** PostGIS Geolocation matching, Recycler discovery, Waste listing management, and Trade request workflows.
- **Iteration 3 (Weeks 10–11):** Virtual tree gamification ("Green Points"), Eco Facts, Leaderboards, Admin Dashboard, and AI Misclassification feedback loop.

Verification, Validation & Freeze Phase (Weeks 12–13):

- Execute System Testing, Integration Testing, Performance/Latency verification, and User Manual writing.
- Final freeze of source code and deployment artifacts.

Project Closing Phase (Week 14):

- Finalize Capstone Project Report (Report 7) and conduct final presentation defense.

2. Quality Management

To achieve the project metrics defined in Section I (such as ≥ 85% AI top-1 classification accuracy, web page load time ≤ 2.0 s, and zero high-severity defects), the team enforces quality controls across four core dimensions:

2.1 Defect Prevention & Code Standards

- **Clean Architecture Compliance:** Enforce strict separation of layers in the .NET solution (Domain, Application, Infrastructure, WebApi) so that domain logic remains independent of external frameworks.
- **Code Reviews:** Every Pull Request (PR) merged into the main branch requires at least one code review and approval by the Tech Lead (Thang) or Sub-leader (Giang).
- **Static Analysis:** Apply standard linter tools (Roslyn analyzers for C# .NET and PEP8 / Flake8 for Python FastAPI).

2.2 Testing Levels & Validation

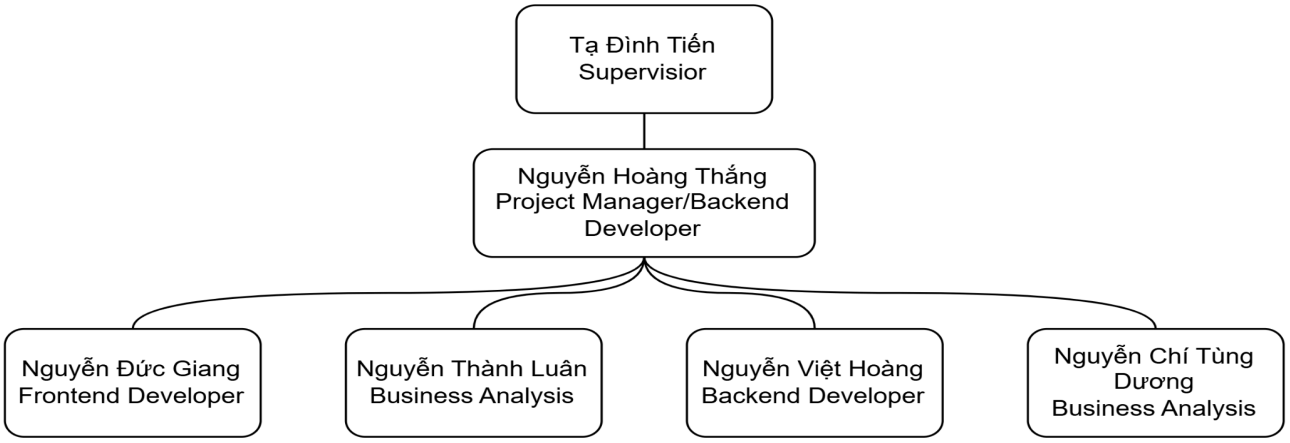
- **Unit Testing:** Write unit tests for business handlers, CQRS services, and rule mapping calculations using xUnit (.NET).
- **Integration Testing:** Test API endpoints, EF Core database mappings, PostGIS spatial queries, and inter-service communication between .NET Web API and Python FastAPI.
- **AI Model Validation:** Evaluate model accuracy on a dedicated test dataset using Precision, Recall, F1-Score, and Confusion Matrix analysis. Enforce an Out-of-Distribution threshold ($< 60\%$ confidence) to trigger re-capture prompts for invalid images.
- **Offline Backup Validation:** Maintain a converted lightweight model (TensorFlow.js / TFLite) as a client-side fallback during live demonstrations to mitigate network outage risks.

3. Project Training Plan

To bridge skill gaps and ensure all team members can execute their assigned work packages efficiently, the following training activities are planned during the initial weeks of the project:

Training Area	Participants	When, Duration
.NET 10 SDK & Clean Architecture	Thang (Leader), Giang, Luan, Hoang, Duong	Week 1 – Week 2 (10 hours)
Python FastAPI & YOLOv8 Computer Vision	Thang (Leader)	Week 2 – Week 3 (12 hours)
PostgreSQL & PostGIS Spatial Querying	Thang, Giang	Week 2 – Week 3 (8 hours)
Git / GitHub Workflow & Docker Compose	Entire Team (5 members)	Week 1 (4 hours)
Figma UI/UX & Responsive Web Design	Giang (Sub-leader)	Week 2 (6 hours)

4. Responsibility Assignments



RACI Chart: R~Responsible, A~Accountable, C~Consulted, I~Informed

Responsibility	THANGNH	GIANGND	LUANNT	HOANGNV	DUONGNCT
Project Planning & Tracking	R	A	I	I	I
Prepare Project Introduction Document	C	A	R	R	R
Prepare SRS Document (Overview Part)	A	C	R	R	R
Prepare SRS Document (User Requirements)	A	C	R	R	R
System Architecture Design	R	A	I	C	I
Database Design	R	A	C	C	C
Backend Development	R	A	C	R	C
Frontend Development	A	R	I	I	I
Integration & API Development	R	A	I	R	I
Test Case Preparation	R	A	I	I	R

Unit Testing	R	A	I	R	I
Integration Testing	A	R	I	R	I
System Testing	A	R	R	R	I
Deployment & Maintenance	R	A	I	I	I

5. Project Communications

Communication Item	Who/ Target	Purpose	When, Frequency	Type, Tool, Method(s)
Daily Sync Meeting	Entire Team (5 members)	Track daily progress, identify blockers, and align daily task assignments.	Mon – Fri (60 mins, 21:30)	Online (Messenger Group / Zalo / Discord)
Weekly Progress Meeting	Entire Team (5 members)	Review weekly deliverables, update Weekly Status Reports, and plan tasks for the next week.	Weekly (Thu 08:00 / Fri 21:00)	Hybrid (Offline Room DE418 / Google Meet)
Supervisor Progress Review	Team & Supervisor (Mr. Ta Dinh Tien)	Report milestone progress, demo prototypes, submit report drafts, and receive feedback.	Bi-weekly / Milestone checkpoints	Offline Meeting (Room DE418, FPT University)

6. Configuration Management

6.1 Document Management

Centralized Storage: All project documents (proposal, SRS, SDS, weekly reports, user guides) are stored and collaborated on via Google Drive (SEP490_G141_FA26).
File Naming Convention: Documents follow the strict naming structure:
[Document_Name]_[Group_ID]_[Version]_[Date]
Example: Report1_SEP490_G141_v1.0.docx, Report2_Project_Tracking_SEP490_G141.xlsx.
Version Control Rules:

- v0.x: Draft versions created during team working sessions.
- v1.0, v2.0: Finalized versions approved by the Leader (Thang) and submitted to the Lecturer.

Audit Trail: Any modification to milestone documents must be recorded in the **Record of Changes / Change Log** table at the beginning of each report.

6.2 Source Code Management

Version Control System: Source code is hosted on GitHub across two dedicated repositories:

- Backend & AI Service: VPGreenLight/intelligent-waste-sorting-trading
- Frontend Web: VP GreenLight/ecolink-web

Branching Strategy (Git Flow):

- main: Production-ready code; protected branch requiring approval from Tech Lead (Thang) or Sub-leader (Giang).
- develop: Integration branch where features are combined and tested.

- feature/[feature-id]: Working branch for specific use cases (e.g., feature/FE-01-ai-scan, feature/FE-06-postgis-matching).
- bugfix/[bug-id]: Branch for resolving issues identified during testing.

Pull Request (PR) & Commit Rules:

- Commit messages must follow the format: [FE-xx/UC-xx] <type>: <description> (e.g., [FE-01] feat: integrate YOLOv8 classification endpoint).
- Every Pull Request requires at least one code review and approval from Leader/Sub-leader before merging into develop or main.

6.3 Tools & Infrastructures

Category	Tools / Infrastructure
Technology	React.js & Tailwind CSS (Frontend Web), ASP.NET Core 10.0 (Backend Clean Architecture), Python FastAPI (Dedicated AI Vision Service)
Database	PostgreSQL 16 + PostGIS Extension (Spatial GPS Queries), Redis Cache, MinIO Object Storage
IDEs / Editors	Visual Studio 2022, Visual Studio Code, PyCharm
Diagramming	Draw.io, StarUML
Documentation	Google Docs, Google Sheets, Google Slides, MS Office
Version Control	GitHub (Source Code), Google Drive (Documents & Reports)
Deployment Server	Docker Compose (Localhost & Cloud VPS Containerization)
Project Management	Google Sheets (Project Tracking & Weekly Reports), GitHub Projects, ClickUp